FoodShare – Backend Schema

Oct 22, 2026 · @momo

Data model overview

Two core collections carry the whole workflow: `users` and `listings`. A listing references the donor who created it, the receiver who claimed it and the volunteer who delivers it. An optional `notifications` collection supports in-app alerts.

From	Field	To	Meaning
listings	donor	users	Who posted the surplus (role donor)
listings	claimedBy	users	Receiver that claimed it (role receiver)
listings	volunteer	users	Volunteer who picked it up (role volunteer)
notifications	user	users	Recipient of the alert
notifications	listing	listings	Listing the alert is about

Collection schemas

users

Field	Type	Rules
name	String	Required
email	String	Required, unique, lowercase
password	String	Required, min 6, bcrypt hash, excluded from queries by default
role	String	Enum: donor, volunteer, receiver
organization	String	Optional; restaurant or NGO name
phone	String	Optional
createdAt, updatedAt	Date	Automatic

listings

Field	Type	Rules
donor	ObjectId (users)	Required
title	String	Required
description	String	Optional
category	String	Enum: cooked, bakery, produce, packaged, dairy, other
quantityKg	Number	Required, at least 0.1
expiresAt	Date	Required
address	String	Optional
location	GeoJSON Point	`coordinates` = [longitude, latitude], required
status	String	Enum: available, claimed, in_transit, delivered, cancelled; default available
claimedBy	ObjectId (users)	Set on claim

Field	Type	Rules
volunteer	ObjectId (users)	Set on pickup
claimedAt, pickedUpAt, deliveredAt	Date	Set on each transition
createdAt, updatedAt	Date	Automatic

notifications (optional, v1.1)

Field	Type	Rules
user	ObjectId (users)	Recipient
listing	ObjectId (listings)	Subject
type	String	claimed, picked_up, delivered, new_pickup
readAt	Date	Null until read

Indexes

Collection	Index	Why
listings	`{ location: "2dsphere" }`	Required for `$near` radius queries
listings	`{ status: 1 }`	Filter the feed and pickup board
listings	`{ donor: 1, createdAt: -1 }`	Donor history
listings	`{ claimedBy: 1 }`, `{ volunteer: 1 }`	Receiver and volunteer history
listings	`{ status: 1, deliveredAt: -1 }`	Analytics over delivered items
users	`{ email: 1 }` unique	Login and duplicate check
notifications	`{ user: 1, readAt: 1 }`	Unread count

Optional: a TTL or scheduled job to mark past-expiry available listings as cancelled.

Key queries

Nearby available listings within a radius, nearest first:

```
Listing.find({
  status: 'available',
  expiresAt: { $gt: new Date() },
  location: {
    $near: {
      $geometry: { type: 'Point', coordinates: [lng, lat] },
      $maxDistance: radiusKm * 1000
    }
  }
}).limit(100)
```

Atomic claim. The status filter makes the update succeed for only one caller:

```
Listing.findOneAndUpdate(
  { _id: id, status: 'available', expiresAt: { $gt: new Date() } },
  { $set: { status: 'claimed', claimedBy: receiverId, claimedAt: new Date() } }
},
  { new: true }
)
```

Transition rules

Action	Required current status	Extra condition	New status
Claim	available	Not expired; caller is receiver	claimed
Pick up	claimed	Caller is volunteer	in_transit
Deliver	in_transit	Caller is the assigned volunteer	delivered
Cancel	available	Caller is the donor owner	cancelled

A `null` result means the listing is missing or in another state, and the API answers 409.

Impact aggregation

One `$facet` pipeline over delivered listings returns every dashboard block in a single round trip:

```
Listing.aggregate([
  { $match: { status: 'delivered' } },
  { $facet: {
    totals: [{ $group: { _id: null, deliveries: { $sum: 1 }, totalKg: { $sum:
'$quantityKg' } } }],
    byCategory: [{ $group: { _id: '$category', kg: { $sum: '$quantityKg' },
count: { $sum: 1 } } }, { $sort: { kg: -1 } }],
    daily: [
      { $match: { deliveredAt: { $gte: since } } },
      { $group: { _id: { $dateToString: { format: '%Y-%m-%d', date:
'$deliveredAt' } }, kg: { $sum: '$quantityKg' } } },
      { $sort: { _id: 1 } }
    ],
    topDonors: [
      { $group: { _id: '$donor', kg: { $sum: '$quantityKg' } } },
      { $sort: { kg: -1 } }, { $limit: 5 },
      { $lookup: { from: 'users', localField: '_id', foreignField: '_id', as:
'donor' } }
    ]
  } }
])
```

Derived metrics (computed in the API from `totalKg`):

```
\text{meals saved} = \frac{\text{totalKg}}{\text{KG\_PER\_MEAL}} \qquad
\text{CO2 avoided} = \text{totalKg} \times \text{CO2\_PER\_KG}
```

Defaults are 0.4 kg per meal and 2.5 kg CO2e per kg; both are environment settings so they can follow a cited source.